

TECHNICAL GUIDE

Linked Lists in Python

A deep dive into dynamic memory structures, node management, and efficient data handling.

Series: Master Data Structures

Provider: CODEHIVE

Contents

01. The Anatomy of a Node	3
02. Linked Lists vs. Arrays	4
03. Structural Variations	5
04. Traversal & Searching	6
05. Deletion Logic	7
06. Insertion Algorithms	8
07. Time Complexity Analysis	9
08. Strategic Summary	10

01. The Anatomy of a Node

A Linked List is a linear data structure where elements are not stored at contiguous memory locations. Instead, elements are linked using pointers.

What is a Node?

Every element in a linked list is a **Node**. A node typically consists of two parts:

1. **Data:** The actual value stored (integer, string, object).
2. **Pointer (Next):** A reference/address to the subsequent node in the sequence.

Concept: Think of it like a scavenger hunt. To find the second item, you must first find the first item, which contains a clue (pointer) telling you where the second item is hidden.

The Head Node

The entry point to any linked list is called the **Head**. If the head is null, the list is empty. The last node in a list points to **null** (None in Python), signifying the end of the chain.

02. Linked Lists vs. Arrays

Understanding when to use a Linked List over an Array is a critical engineering skill. While arrays are built-in, linked lists are often custom-built to leverage specific memory advantages.

Feature	Arrays (Lists)	Linked Lists
Memory Allocation	Fixed/Static size	Dynamic (Grows at runtime)
Storage	Contiguous blocks	Non-contiguous (Random)
Access Time	$O(1)$ - Random Access	$O(n)$ - Sequential Access
Insertion/Deletion	$O(n)$ - Requires Shifting	$O(1)$ - Pointer Update Only
Memory Efficiency	High (No pointers)	Lower (Pointer overhead)

Why choose Linked Lists?

- **No Shifting:** Adding an item to the middle of an array requires moving every subsequent item. In a linked list, you just change two pointers.
- **Flexibility:** You don't need to know the size of your data beforehand.

03. Structural Variations

Depending on the directional needs of your application, linked lists come in three flavors.

1. Singly Linked List

The simplest form. Each node points only to the next node. Navigating backward is impossible without restarting from the head.

2. Doubly Linked List

Each node contains two pointers: next and previous.

Benefit: Can be traversed in both directions.

Cost: Higher memory consumption (2 pointers per node).

3. Circular Linked List

The last node points back to the head node instead of null.

Use Case: Perfect for iterative tasks like round-robin scheduling or media players (looping a playlist).

04. Traversal & Searching

To perform any action (finding a value, counting items), you must "walk" through the list.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def traverseAndPrint(head):
    currentNode = head
    while currentNode:
        print(currentNode.data, end=" → ")
        currentNode = currentNode.next
    print("null")
```

Finding the Minimum Value

Since we cannot jump to an index, we track the minimum while iterating.

```
def findLowestValue(head):
    minValue = head.data
    currentNode = head.next
    while currentNode:
        if currentNode.data < minValue:
            minValue = currentNode.data
        currentNode = currentNode.next
    return minValue
```

05. Deletion Logic

The key to deleting a node is **bridging the gap**. If you simply delete a node, the reference to the rest of the list is lost forever.

The Strategy: Identify the node before the one you want to delete. Change its next pointer to point to the deleted node's next sibling.

```
def deleteSpecificNode(head, nodeToDelete):  
    # Case 1: Deleting the Head  
    if head == nodeToDelete:  
        return head.next  
  
    currentNode = head  
    # Traverse to find the node BEFORE the target  
    while currentNode.next and currentNode.next != nodeToDelete:  
        currentNode = currentNode.next  
  
    if currentNode.next is not None:  
        # Bridge the gap  
        currentNode.next = currentNode.next.next  
  
    return head
```

06. Insertion Algorithms

Insertion requires careful pointer management to ensure the chain remains intact. Timing is everything: you must link the new node to the future sibling before linking the previous sibling to the new node.

```
def insertNodeAtPosition(head, newNode, position):
    # Inserting at the Head
    if position == 1:
        newNode.next = head
        return newNode

    currentNode = head
    # Navigate to index - 1
    for _ in range(position - 2):
        if currentNode is None: break
        currentNode = currentNode.next

    # Connect new node to the chain
    newNode.next = currentNode.next
    # Redirect previous node to the new node
    currentNode.next = newNode
    return head
```


07. Time Complexity Analysis

Big O Notation help us compare the scalability of Linked Lists against Arrays. While they share some traits, their strengths are polar opposites.

Operation	Linked List	Array / Python List
Index Access	$O(1)$	$O(n)$ - Must traverse
Insert/Delete (Start)	$O(1)$	$O(n)$ - All items shift
Insert/Delete (End)	$O(1)$ if Tail kept	$O(1)$ - Amortized
Search (Linear)	$O(n)$	$O(n)$

Note on Binary Search: Binary search is **impossible** with standard linked lists because you cannot jump to the middle element in $O(1)$ time.

08. Strategic Summary

Linked lists are the foundation for more complex data structures like Stacks, Queues, and Graphs. Their ability to manage memory efficiently makes them ideal for systems where data size is unpredictable.

Key Takeaways

- **Nodes are Independent:** They live anywhere in memory, unlike array elements.
- **Pointers are Vital:** Losing a pointer means losing data.
- **Direction Matters:** Choose Singly for memory efficiency or Doubly for flexibility.
- **Traversal is the Cost:** Most operations require $O(n)$ traversal before the $O(1)$ pointer change can happen.

Congratulations! You have completed the Linked List Module.

(c) 2026 CODEHIVE Tutorials. All rights reserved.